

認識 Common Type System

- **13-1:什麼是CTS**

什麼是CTS (Common Type System).....	13-1
CTS 的類型.....	13-2

- **13-2:Value 型態 (實值型別)與Reference 型態 (參考型別)**

實值型別的特性.....	13-3
實值型別一覽.....	13-4
參考型別的特性.....	13-5
參考型別一覽.....	13-6
Value型態 vs Reference型態.....	13-7

Module 13

什麼是CTS (Common Type System)

- **CTS是 .NET 架構中的通用型別系統，用來統一所有 .NET 語言所使用的資料型態，以確保跨語言間能正確互通與執行。**
- **CTS 的主要功能：**
 - 建立跨語言的執行架構，支援高效能程式碼執行。
 - 提供物件導向模型，讓多種 .NET 語言能以一致方式實作。
 - 定義各語言在處理型別時必須遵循的規則。提供基本型別程式庫（如 Boolean、Byte、Char、Int32 等）。
 - 與 CLS (Common Language Specification) 通用語言規範 搭配使用，以確保不同語言間的相容性與一致性。
- **CTS 是 .NET 跨語言整合的基礎，讓 C#、VB.NET、F# 等語言能共享相同的型別系統與執行規則。**

CTS 的類型

- 在程式執行時，資料必須先儲存在記憶體中才能被存取。C# 根據資料在記憶體中的儲存方式，將型別分為兩大類：
 - 實值型別(Value Type)：
 - 內建型態(簡單型態)
 - 一般數值型態、布林值、Unicode字元。
 - 結構型態(struct)
 - 列舉型態(enum)
 - 參考型別(Reference Type)

實值型別的特性

- 變數名稱的記憶位址中直接存放資料本身。
- 建立、存取與回收速度快，效能佳。
- 屬於直接存取資料的型別，無需透過參考位址。
- 複製變數時，會產生完整複本。

實值型別一覽

實值型別	說明
簡單型別	帶正負號的整數： <code>sbyte</code> 、 <code>short</code> 、 <code>int</code> 、 <code>long</code>
	不帶正負號的整數： <code>byte</code> 、 <code>ushort</code> 、 <code>uint</code> 、 <code>ulong</code>
	Unicode字元： <code>char</code>
	IEEE浮點數： <code>float</code> 、 <code>double</code>
	高精度十進位數： <code>decimal</code>
	布林值： <code>bool</code>
列舉型別	型式為 <code>enum E { //列舉值... }</code>
結構型別	型式為 <code>struct S { //結構... }</code>
不可為Null的型別	其他所有不包含 <code>null</code> 值的數值型別擴充

參考型別的特性

- 資料儲存在 **Heap** (堆積區) 記憶體中，而 **Stack** (堆疊區) 僅存放資料在 **Heap** 中的記憶體位址。
- 存取資料時，必須透過 **Stack** 中的位址間接取得資料值。
- 屬於間接存取資料的型別。
- 多個變數可指向同一記憶體位址，因此修改其中之一，其他也會受到影響。
- **Heap** 的記憶體配置與釋放由 **CLR** (**Common Language Runtime**) 自動管理。

參考型別一覽

參考型別	說明
類別型別	所有其他型別的最終基底型別： Object
	Unicode 字串： string
	型式為 class C { //... } 的使用者定義型別
介面型別	型式為 interface I { //... } 的使用者定義型別
陣列型別	一維及多維，例如： int[] 和 int[,]
委派型別	型式為 delegate int D(//...) 的使用者定義型別

Value型態 vs Reference型態

	Value型態	Reference型態
變數的內容	內容值即資料	存放資料的位置
變數1 = 變數2	指派後，=號左右兩個變數各自擁有的獨立資料	指派後，=號左右兩個的變數存放同一資料的地址
相互指派變數後改變值的影響	對任何一個變數操作不會影響另外一個變數	對任何一個變數操作會影響另外一個變數
宣告建立物件	不須new建立物件	必須new建立物件